

Web technológiák 2

Full-stack Hardver Nyilvántartó Rendszer

Hallgató: Dobozi Botond

Neptun-kód: HYS4P5

Tárgy: Web technológiák 2

Félév: 2025/26 2. félév

Projektjegyzőkönyv

2026. május 11.

Tartalomjegyzék

1. Bevezetés	2
1.1. A projekt fő célkitűzései	2
2. Alkalmazott technológiák	3
2.1. Backend technológiák	3
2.2. Frontend technológiák	3
3. Rendszerarchitektúra és Mappaszerkezet	4
3.1. Backend struktúra	4
3.2. Frontend struktúra	4
4. Adatmodell és Adatbázis	5
4.1. Termékmodell (Product)	5
4.2. Üzenetmodell (Message)	5
5. Frontend implementáció	6
5.1. Dinamikus útválasztás	6
5.2. Komponens architektúra	6
6. Állapotkezelés a gyakorlatban	7
7. Felhasználói felület és UX	8
8. Telepítés és Futtatás	9
8.1. Backend indítása	9
8.2. Frontend indítása	9
9. Összegzés	10

1. fejezet

Bevezetés

A projekt a Web technológiák 2 tantárgy keretében készült el. A cél egy olyan modern, full-stack webalkalmazás kifejlesztése volt, amely megfelel a mai iparági elvárásoknak: szétválasztott kliens- és szerveroldali architektúra, típusbiztonság és skálázható adatkezelés.

A fejlesztés során a **TypeScript** nyelvet választottam mind a frontend, mind a backend oldalon, mivel a statikus típusellenőrzés jelentősen csökkenti a futásidejű hibák számát és javítja a kód olvashatóságát.

1.1. A projekt fő célkitűzései

- Dinamikus termékkatalógus megvalósítása.
- CRUD (Create, Read, Update, Delete) műveletek biztosítása az adminisztrációs felületen.
- Valós idejű állapotkezelés a kliensoldalon.
- Kapcsolati űrlap és üzenetküldési funkció integrálása adatbázis háttérrel.

2. fejezet

Alkalmazott technológiák

A rendszer architektúrája két fő részre osztható, amelyek REST API-n keresztül kommunikálnak egymással.

2.1. Backend technológiák

- **Node.js & Express.js:** A szerveroldali futtatókörnyezet és keretrendszer.
- **MongoDB & Mongoose:** NoSQL adatbázis a rugalmas sémakezelés érdekében.
- **TypeScript:** Szigorú típusosság a backend logikában.

2.2. Frontend technológiák

- **Next.js (App Router):** A modern React keretrendszer, amely támogatja a szerveroldali renderelést (SSR).
- **Tailwind CSS:** Utility-first CSS keretrendszer a gyors és rezponzív dizájnhoz.
- **Context API & useReducer:** Globális állapotkezelés a komplex interakciókhoz (pl. kosár kezelése).

3. fejezet

Rendszerarchitektúra és Mappaszerkezet

A projekt felépítése követi a "Separation of Concerns" elvét. A kliens és a szerver kódja elkülönül, így a komponensek függetlenül fejleszthetők és tesztelhetők.

3.1. Backend struktúra

A `backend/src` könyvtár moduláris felépítésű:

- **models/**: Az adatbázis sémák definíciói (Product, Message).
- **controllers/**: A HTTP kérések logikai feldolgozása.
- **services/**: Az adatbázis-műveletek tényleges végrehajtása.
- **routes/**: Az API végpontok deklarálása.

3.2. Frontend struktúra

A `frontend/app` könyvtár a Next.js App Router logikáját követi:

- **(pages)/**: Statisztikák, Kapcsolat, Nyilvántartás és Admin felületek.
- **components/**: Újrahasznosítható UI elemek (Navbar, Footer, StatCard).
- **context/**: A globális állapotot (state) kezelő fájlok.

4. fejezet

Adatmodell és Adatbázis

Az alkalmazás MongoDB adatbázist használ, amelyet a `db.ts` fájl konfigurál.

4.1. Termékmodell (Product)

A termékek tárolásáért felelős séma az alábbi főbb mezőket tartalmazza: név, ár, leírás, kategória és kép elérési útja. Ez teszi lehetővé a webshop funkcióinak kiszolgálását.

4.2. Üzenetmodell (Message)

A kapcsolati űrlapon beküldött adatok tárolására szolgál, tartalmazva a küldő nevét, e-mail címét és az üzenet tartalmát.

```
1 const ProductSchema = new Schema({  
2   name: { type: String, required: true },  
3   price: { type: Number, required: true },  
4   description: String,  
5   category: String  
6 });
```

Listing 4.1. Mongoose séma példa

5. fejezet

Frontend implementáció

5.1. Dinamikus útválasztás

A Next.js lehetővé teszi, hogy minden funkció saját mappát kapjon az `app` könyvtárban. Például a `/nyilvantartas` útvonal felelős a terméklista megjelenítéséért, míg az `/admin` a szerkesztési funkciókért.

5.2. Komponens architektúra

A `StatCard.tsx` és `ProductCard.tsx` komponensek biztosítják az adatok konzisztens megjelenítését. A Tailwind CSS használatával minden komponens reszponzív, így mobil eszközökön is használható a felület.

6. fejezet

Állapotkezelés a gyakorlatban

Az alkalmazás egyik legfontosabb része a `GlobalContext`. Ez teszi lehetővé, hogy az API-ból lekért adatok (például a termékek listája) az egész alkalmazásban elérhetőek legyenek anélkül, hogy újra le kellene kérni őket.

A `Reducer.tsx` kezeli az állapotátmeneteket, például:

- `SET_PRODUCTS`: Adatok betöltése a backendről.
- `ADD_PRODUCT`: Új termék hozzáadása a listához.
- `DELETE_PRODUCT`: Termék eltávolítása.

7. fejezet

Felhasználói felület és UX

A dizájn során törekedtem a tisztaságra és az átláthatóságra. A főoldalon található statisztikai kártyák (StatCard) azonnali visszajelzést adnak a készlet állapotáról. A navigáció (Navbar) minden aloldalon rögzített, megkönnyítve a közlekedést a funkciók között.

8. fejezet

Telepítés és Futtatás

A projekt futtatásához Node.js környezet és egy futó MongoDB példány szükséges.

8.1. Backend indítása

```
1 cd backend  
2 npm install  
3 npm run dev
```

8.2. Frontend indítása

```
1 cd frontend  
2 npm install  
3 npm run dev
```

9. fejezet

Összegzés

A fejlesztés során sikerült egy teljes körű full-stack alkalmazást létrehozni, amely lefedi a modern webfejlesztés minden fontos területét. A TypeScript és a React/Next.js párosítása kiváló fejlesztői élményt és stabil végeredményt biztosított. A jövőben a rendszer bővíthető lenne autentikációval és fizetési modul integrációjával.